A BENCHMARKING INITIATIVE FOR AUDIO-DOMAIN MUSIC GENERATION USING THE FREESOUND LOOP DATASET

Tun-Min Hung[1,2] **Bo-Yu Chen**[1] **Yen-Tung Yeh**[1,2] **Yi-Hsuan Yang**[1,3]

[1] Academia Sinica, [2] National Taiwan University, [3] Taiwan AI Labs

r09946015@ntu.edu.tw, bernie40916@gmail.com, b06611042@ntu.edu.tw, yang@citi.sinica.edu.tw

ABSTRACT

This paper proposes a new benchmark task for generating musical passages in the audio domain by using the drum loops from the FreeSound Loop Dataset, which are publicly re-distributable. Moreover, we use a larger collection of drum loops from Looperman to establish four model-based objective metrics for evaluation, releasing these metrics as a library for quantifying and facilitating the progress of musical audio generation. Under this evaluation framework, we benchmark the performance of three recent deep generative adversarial network (GAN) models we customize to generate loops, including StyleGAN, StyleGAN2, and UNAGAN. We also report a subjective evaluation of these models. Our evaluation shows that the one based on StyleGAN2 performs the best in both objective and subjective metrics.

1. INTRODUCTION

Audio-domain music generation involves generating musical sounds either directly as audio waveforms or as time-frequency representations such as the Mel spectrograms. Besides modeling musical content in aspects such as pitch and rhythm, it has the additional complexity of modeling the spectral-temporal properties of musical sounds, compared to its symbolic-domain music generation counterpart. In recent years, deep learning models have been proposed for audio-domain music generation, starting with simpler tasks such as generating instrumental single notes [1–4], a task also known as *neural audio synthesis*. Researchers have also begun to address the more challenging setting of generating sounds of longer duration [5–12]. For example, Jukebox [11] aims to generate realistic minutes-long singing voices conditioned on lyrics, genre, and artists; and UNAGAN [9] aims to generate musical passages of finite yet arbitrary duration for singing voices, violin, and piano, in an unconditional fashion.

The focus of this paper is on the evaluation of audio-domain music generation. We note that, for model training and evaluation, research on generating single notes quite often adopts NSynth [1], a large public dataset consisting of individual notes from different instruments. The use of a common dataset for evaluation ensures the validity of performance comparison between different models. Such a standardized dataset for benchmarking, however, is not available when it comes to generating longer musical passages, to our best knowledge. Oftentimes private in-house datasets are employed in existing works; for example, both UNAGAN [9] and Jukebox [11] employ audio recordings scrapped from the Internet, which cannot be shared publicly. The only exception is MAESTRO, a public dataset with over 172 hours of solo piano performances, employed by MelNet [8], UNAGAN [9], and MP3net [12]. However, MAESTRO is piano-only so not diverse enough in timbre.

We see new opportunities to address this gap with the recent release of the FreeSound Loop Dataset (FSLD) [15], which contains 9,455 production-ready, public-domain loops distributed under Creative Commons licenses. [1] We therefore propose to use *audio-domain loop generation*, a task seldom reported in the literature, to set a benchmark for musical audio generation research.

We deem loops as an adequate target for audio generation for their following merits. First, loops are audio excerpts, usually of short duration, that can be played in seamless manner [15, 16]. Hence, the generated loops can be played repeatedly. Second, loops are fundamental units

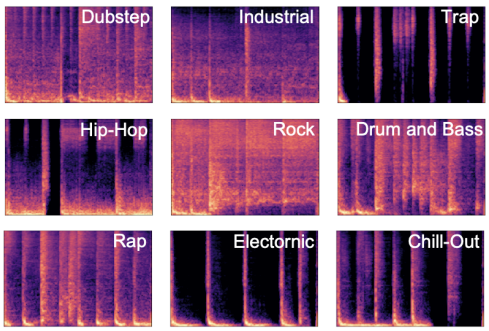


Figure 1. The mel-spectrograms of some random drum loops generated by the StyleGAN2 model [13] trained on the looperman dataset, with the genre labels predicted by the short-chunk CNN [14] classifier (see Section 4.1).

© F. Author, S. Author, and T. Author. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** F. Author, S. Author, and T. Author, "A Benchmarking Initiative for Audio-domain Music Generation using the FreeSound Loop Dataset", in *Proc. of the 22nd Int. Society for Music Information Retrieval Conf.*, Online, 2021.

[1] https://zenodo.org/record/3967852

in the production of many contemporary dance music genres. A loop can usually be associated with a single genre or instrument label [15], and a certain "role" (e.g., percussion, FX, melody, bass, chord, voice) [17]. Third, loops are fairly diverse in their music content and timbre, as sound design has been a central part in making loops.

A primary contribution of this paper is therefore the proposal and implementation of using FSLD as a benchmark for audio generation. In particular, we adapt three recent deep generative adversarial network (GAN) [18] models and train them on the *drum-loop* subset of FSLD, and report thorough evaluation of their performance, both objectively and subjectively. This includes UNAGAN [9] and two state-of-the-art models for image generation, StyleGAN [19] and StyleGAN2 [13].

Drum loop generation is interesting in its own right due to its applications in automatic creation of loop-based music [20]. As [21] indicates, drum beats represent one of the most critical and fundamental elements that form the style of EDM. Moreover, drum loops are already fairly diverse in musical content, as demonstrated in Figure 1. Although we only consider drum loops here for the sake of simplicity, this benchmark can be easily extended to cover all the loops from FSLD in the near future.

Our secondary contribution lies in the development of standardized objective metrics for evaluating audio-domain loop generation, which can be equally important as having a standardized dataset. We collect a larger drum loop dataset from an online library called looperman, [2] with roughly 9 times more drum loops than FSLD, and use this looperman dataset to build four model-based metrics (e.g., inception score [22]) [3] to evaluate the acoustic *quality* and *diversity* of the loops generated by the GAN models. While this looperman dataset cannot be released publicly due to copyright concerns, we release the metrics and the trained GAN models for drum loop generation at the following GitHub repo: `https://github.com/allenhung1025/LoopTest`.

Moreover, we put some of the generated drum loops on an accompanying **demo website**, [4] which we recommend readers to visit and listen to. We also present the result where we use the method of *style-mixing* of StyleGAN2 to generate "interpolated" versions of loops.

Below, we review related work in Section 2, present the datasets in Section 3, the proposed objective metrics in Section 4, the benchmarked models in Section 5, and the evaluation result in Section 6.

2. RELATED WORK

Existing work on audio-domain music generation can be categorized in many ways. First, an **unconditional** audio generation model takes as input a vector $\mathbf{z} \in \mathbb{R}^{N_z}$ of a fixed number of random variables (or a sequence of such vectors; see below) and generates an audio piece from scratch. When side information of the target audio to be generated is available, we can feed such prior information as another vector $\mathbf{c} \in \mathbb{R}^{N_c}$ and use it as an additional input to the generative model, making it a **conditional** generation model. For example, GANSynth [2] uses the pitch of the target audio as a condition. While we focus on unconditional generation in our benchmarking experiments presented in Section 6, it is straightforward to extend all the models presented in Section 5 to take additional conditions.

Second, some existing models can only generate **fixed-length** output, while others can do **variable-length** generation. One approach to realize variable-length generation is by using as input to the generative model a sequence of latent vectors $\mathbf{z}_1, \mathbf{z}_2, \ldots$, instead of just one latent vector $\mathbf{z}$. This is the approach taken by UNAGAN [9], Jukebox [11], and VQCPC-GAN [23].

Third, existing models for generating single notes are typically **non-autoregressive** models [2–4], i.e., the target is generated at one shot. When it comes to generating longer phrases, **autoregressive** models, that generate the target piece one frame or one time sample at a time in the chronological order, might perform better [5, 11], as the output of such models depends explicitly on the previous frames (or samples) that have been generated.

Existing models have been trained and evaluated to generate different types of musical audio, including singing voice [9–11], drum [3, 4, 24, 25], violin [9], and piano [5, 8, 9, 12]. The only work addressing loop generation is the very recent LoopNet model from Chandna *et al.* [26]. They also use loops from looperman but not anything from FSLD or other public datasets, hence not constituting a benchmark for audio generation.

For drum generation in particular, work has been done in the symbolic domain to generate drum patterns [27–30] and a drum track as part of a symbolic multi-track composition [31–33]. For example, DeepDrummer [29] employs human-in-the-loop to produce drum patterns preferred by a user. In the audio domain, DrumGAN [3] and the model proposed by Ramires *et al.* [4] both work on only single hits, i.e., one-shot drum sounds. They both use the Audio Commons models [34] to extract high-level timbral features to condition the generation process. DrumNet [35] is a model that generates a sequence of (monophonic) kick drum hits, not the sounds of an entire drum kit.

3. DATASETS

Two datasets are employed in this work. The first one is a subset of drum loops from the public dataset FSLD [15], which is used to train the generative models for benchmarking. FSLD comes with detailed manual labeling of the loops with tags such as instrumentation, rhythm, tone and genre. As stated in the FSLD paper [15], FSLD is balanced in terms of musical genre. By picking loops which are tagged with the keywords "drum", "drums" or "drum-loop", we are able to find 2,608 drum loops out of the 9,455 loops available in FSLD. We do not need to hold out any of them as test data but use all these bars for training our generative models, since we focus on unconditional generation

[2] `https://www.looperman.com/`

[3] We refer to them as *model-based* metrics because we need to build a classifier or a clustering model to calculate the metrics; see Section 4.

[4] `https://loopgen.github.io/`

in this paper; i.e., each generative model will generate a set of loops randomly for evaluation.

The second dataset is a larger, private collection of drum loops we collect from looperman, a website hosting free music loops. [5] We are able to collect in total 23,983 drum loops, which is much more than the drum loops in FSLD. We use the looperman dataset mainly for establishing the model-based objective metrics for evaluation (see Section 4). For instance, we train an audio-based genre classifier using looperman to set up the drum-loop version of the "inception score" [22, 36] to measure how likely a machine-generated loop sounds like a drum loop. Figure 2 shows the number of tracks per genre tag in looperman, which exhibits a typical long-tail distribution. We can see that "Trap" is the most frequent genre, with 5,903 loops.

We use looperman instead of FSLD to set up such objective metrics, since a larger dataset increases the validity and generalizability of the metrics. Moreover, although we cannot re-distribute the loops from looperman according to its terms, we can share checkpoints of the pre-trained models for computing the proposed objective metrics.

3.1 Data Pre-processing

As we are interested in benchmarking the performance of one-bar loop generation, we perform downbeat tracking using the state-of-the-art recurrent neural network (RNN) model available in the Madmom library [37, 38] to slice every audio file into multiple one-bar loops. [6] After this processing, we have in total 13,666 and 128,122 one-bar samples from FSLD and looperman, respectively. We refer to these two collections of one-bar drum loops as the **freesound** and **looperman** datasets hereafter. We note that all these one-bar samples are of four beats.

As shown in Figure 3, the one-bar samples in either the freesound or looperman datasets have different tempos and hence different lengths. To unify their length to facilitate benchmarking, we use pyrubberband [7] to temporally stretch each of them to 2-second long, namely to have 120 BPM (beat-per-minute) as their tempo. We listened to some of the stretched samples in both datasets and found most sounded plausible with little perceptible artifacts. [8]

All the loops are in 44,100 Hz sampling rate. We downmix the stereo ones into mono. After that, we follow the setting of UNAGAN [9] to compute the Mel spectrograms of these samples, with 1,024-point window size hann window and 275-point hop size for short-time Fourier Transform (STFT), and 80 Mel channels.

[5] As stated on `https://www.looperman.com/help/terms`, "All samples and loops are free to use in commercial and non commercial projects." But, "You may NOT use or re-distribute any media from the loops section of looperman.com as is either for free or commercially on any other web site." (Accessed August 1, 2021)

[6] The downbeat tracker in Madmom is fairly accurate for percussive audio such as the drum loops. For example, it reaches F1-score of 0.863 on the Ballroom dataset [39], according to [38].

[7] `https://pypi.org/project/pyrubberband/`

[8] This, however, may not be the case if the loops are not drum loops. Some data filtering might be needed then, e.g., to remove those whose tempo are much away from 120 BPM.

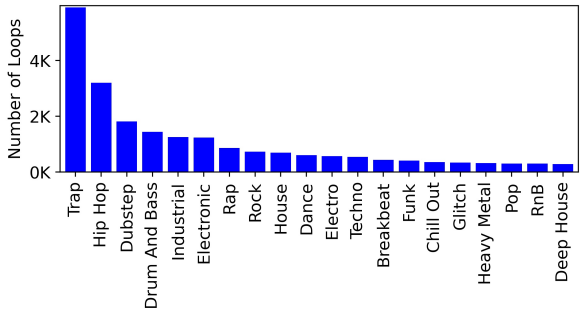


Figure 2. Genre distribution of the drum loops from looperman; we display only the top 20 out of 66 genres.

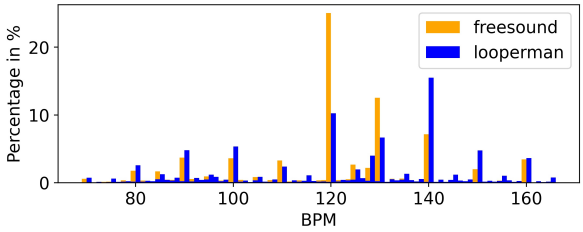


Figure 3. Tempo distribution of the two sets of loops. Y axis represents the percentage of all loops in the dataset.

4. EVALUATION METRICS

We consider four metrics in our benchmark, developing the drum-loop version of them using the looperman dataset.

4.1 Inception Score (IS)

IS [22, 36] measures the quality of the generated data and detects whether there is a mode collapse by using a pre-trained domain-specific classifier. It is computed as the KL divergence between the conditional probability $p(y|x)$ and marginal probability $p(y)$,

$$\text{IS} = \exp\left(E_x[\text{KL}(p(y|x)\|p(y)]\right), \tag{1}$$

where $x \in \mathcal{X}$ denotes a data example (e.g., a generated loop), and $y \in \mathcal{Y}$ is a pre-defined class. Specifically, the calculation of IS involves building a classifier over the type of data of interest, and it achieves high score (namely, the *higher* the better) score when 1) each of the generated data can be classified to any of the predefined classes with high confidence, and 2) the generated data as a whole has close to uniform distribution over the predefined classes.

We use looperman to establish such a classifier, using its genre labels for training a 66-class classifier over the Mel spectrograms of one-bar samples. Specifically, we split the data by 100,000/10,000/18,111 as the training, validation, and test sets, and use the state-of-the-art music auto-tagging model *short-chunk CNN* [14] [9] for model training. The classifier achieves 0.748 accuracy on the test set.

4.2 Fréchet Audio Distance (FAD)

The idea of FAD, as proposed by Kilgour *et al.* [40], is to measure the closeness of the data distribution of the real

[9] `github.com/minzwon/sota-music-tagging-models`

data versus that of the generated data, in a certain embedding space. Specifically, they pre-train a VGGish-based audio classifier on a large collection of YouTube videos for classifying 300+ audio classes and sound events, and then use the second last 128-dimension layer (i.e., prior to the final classification layer) for this embedding space [40]. The data distributions of $r$eal and $g$enerated data in this space are modeled as a multi-variate normal distribution characterized by $(\mu_r, \mathbf{\Sigma}_r)$ and $(\mu_g, \mathbf{\Sigma}_g)$ respectively. The FAD score is then computed by the following equation,

$$\text{FAD} = \|\mu_r - \mu_g\|^2 + tr(\mathbf{\Sigma}_r + \mathbf{\Sigma}_g - 2\sqrt{\mathbf{\Sigma}_r\mathbf{\Sigma}_g}), \quad (2)$$

and is the *lower* the better (down to zero). We use the open source code and pre-trained classifier [10] to compute the FAD, using the looperman data as the real data and the output of a generative model as the generated data.

4.3 Diversity Measurement

Following [9], we measure diversity with the number of statistically-different bins (NDB) and Jensen-Shannon divergence (JSD) metrics proposed by Richardson *et al.* [41], via the official open source code. [11] We firstly run K-means clustering over normalized Mel spectrograms of 10 thousands one-bar samples randomly picked from looperman to get $K = 100$ clusters, and count the number of samples per cluster, n_k, for each k. Then, given a collection of loops randomly generated by a generative model, we fit the loops into the clustering and also count the number of fitted samples per cluster, $\widehat{n_k}$. We can then measure the difference between the two distributions $\{n_k\}$ and $\{\widehat{n_k}\}$ by either the number of statistically-different bins (among the K bins; the *lower* the better) and their JSD (the *lower* the better; down to zero). Richardson *et al.* [41] recommend reporting the value of NDB divided by K, saying that if the two samples do come from the same distribution, NDB$/K$ should be equal to the significance level of the statistical test, which we set to 0.05.

5. BENCHMARKED GENERATIVE MODELS

We develop and evaluate in total three recent deep generative models, all of which happen to be GAN-based [18]. The first model is **StyleGAN2** [13], which represents the state-of-the-art in image generation, included here intending to test its applicability for musical audio generation (which has not been reported elsewhere, to our best knowledge). The second model, **StyleGAN** [19], is a precursor of StyleGAN2, tested on spoken digit generation before (akin to single note generation in music) [43] but not on musical audio generation. Both StyleGAN and StyleGAN2 generate only fixed-length output, which is fine here since our samples have constant length. The last model, **UNAGAN** [9], represents a state-of-the-art in musical audio generation, capable of generating variable-length output. For fair comparison, we only require UNAGAN to generate two-second samples as the other two. Schematic plots of the three models can be found in Figure 4.

All these three models are trained to generate Mel spectrograms, with phase information missing. But, the Mel spectrograms can later be converted into audio waveforms by a separate *neural vocoder*, such as WaveNet [5], WaveGlow [44], DiffWave [45], or MelGAN [46]. We are in favor of MelGAN for it is non-autoregressive and therefore fast in inference time, and for there is official open source code that is easy to use. [12] We train MelGAN on the looperman dataset and use it in all our experiments.

5.1 StyleGAN

StyleGAN and StyleGAN2 are both non-autoregressive models for generating images. They take a constant tensor of size $4 \times 4 \times 512$ as input, and use a mapping network $f(\cdot)$ consisting of eight linear layers to map a random latent vector $\mathbf{z}$ to an intermediate lantent vector $\mathbf{w}$, which affects the generation process by means of adaptive instance normalization (AdaIN) operations in every block of the generator [19]. Each bock progressively upsamples its input to a larger tensor, until reaching the target size of 1024×1024 by the end with in total eight such blocks.

The input tensor of StyleGAN and StyleGAN2 can be interpreted as 512 4×4 tiny images. This tensor is learned and then fixed during the inference stage while generating new images, using different $\mathbf{z}$ each time. We modify it to be a $5 \times 20 \times 512$ tensor in our work, to generate a 80×320 Mel spectrogram through four upsampling blocks.

Our implementation of StyleGAN is based on an open source code. [13] For model training, StyleGAN employs the non-saturating loss with R_1 regularization [47] and a progressive-growing training strategy [42]. We use 0.9 mixing regularization ratio [42], and set the batch size to 32, 16, 8, 4 in the respective scale, from low to high resolution. In every scale, we train with 1.2M samples. We deployed Adam optimization algorithm and set the learning rate to 1e–3. The total training time is 120 hr on an NVIDIA GTX1080 GPU with 8GB memory.

5.2 StyleGAN2

StyleGAN2 [13] is an improved version of StyleGAN with many structural changes, including replacing AdaIN by a combination of "modulation" and "demodulation" layers, processing the input tensor differently, adding the Gaussian noise outside of the style blocks etc. The weights in the 3×3 convolution layers are scaled with $f(\mathbf{z})$ in the Modulation block and normalized by L2 norm in the DeModulation block. We refer readers to the original paper [13] for details. Our implementation of StyleGAN2 is based on another open source code, [14] with similar training strategies as the StyleGAN case, but two times larger learning rate, no progressive growing, and a constant batch size of 8 for 1M samples. The total training time is 100 hr on a GTX1080.

[10] github.com/google-research/google-research/tree/master/frechet_audio_distance

[11] github.com/eitanrich/gans-n-gmms

[12] github.com/descriptinc/melgan-neurips

[13] github.com/rosinality/style-based-gan-pytorch

[14] github.com/rosinality/stylegan2-pytorch

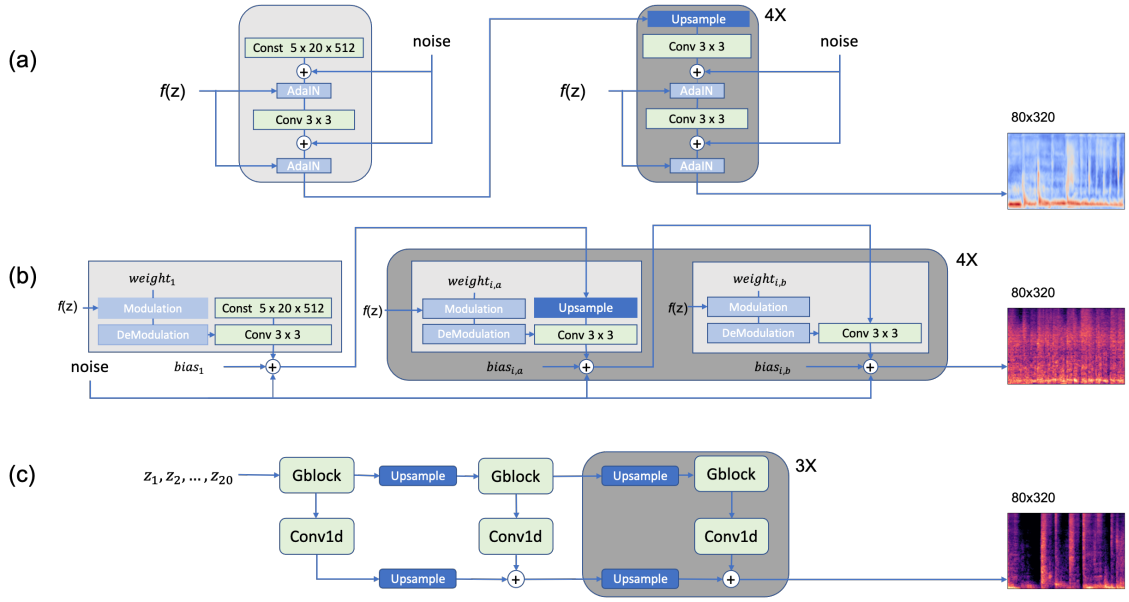


Figure 4. Schematic plots of the adapted (a) StyleGAN [19], (b) StyleGAN2 [42], and (c) UNAGAN [9] in our benchmark. Only a single latent vector $\mathbf{z}$ is used as the input for the fully convolutional models in (a) and (b), while a sequence of 20 latent vectors are used in model (c), which uses a stack of gate recurrent unit (GRU) layer and grouped convolution layer in each of its 'Gblocks' [9, 10]. In (b), weight$_i$ and bias$_i$ are parameters of the 3×3 convolution layers to be learned.

5.3 UNAGAN

UNAGAN [9] is a non-autoregressive model originally designed for generating variable-length singing voices in an unconditional fashion. The authors also demonstrate its effectiveness in learning to generate passages of violin, piano, and speech. What makes UNAGAN different from existing models such as StyleGAN, WaveGAN [48], DrumGAN [3], and GANSynth [2] is that UNGAN takes a sequence of latent vectors $\mathbf{z}_1, \mathbf{z}_2, \ldots$ as input, instead of just a single one. This sequence of latent vectors, together with the recurrent units inside its 'Gblocks' [9,10] (see Figure 4(c)), facilitates UNAGAN to generate variable-length audio with length proportional to the length of the input latent sequence. UNAGAN adopts a hierarchical architecture that generates Mel spectrograms in a coarse-to-fine fashion similar to the progressive upsampling blocks in StyleGAN and StyleGAN2. UNAGAN uses the BEGAN-based adversarial loss [49], and an additional cycle consistency loss [50] to stabilize training and for increasing diversity. Our implementation of UNAGAN is based on the official open source code. [15] We fix the number of input latent vectors to 20 and train the model with Adam, 1e–4 learning rate, and a batch size of 16 for 100k iterations, amounting to 40 hr on a GTX1080.

6. EVALUATION

6.1 Objective Evaluation Result

Table 1 presents the objective evaluation result of models trained on the *freesound* dataset. Each model generates 2,000 random loops to compute the scores. We also compute these metrics on the two real datasets and add the results to Table 1, to offer an oracle reference. We see that the IS of StyleGAN2 is the closest to that of the freesound dataset, followed by UNAGAN and then StyleGAN. Student's t-test shows that the performance edge of StyleGAN2 over either UNAGAN or StyleGAN is statistically significant (p-value$<$0.01). This reveals the efficacy of StyleGAN2 for generating fixed-length audio.

The scores in JS and NDB further support the superiority of StyleGAN2, showing that its output is the most diverse among the three.

The scores in FAD, however, shows that UNAGAN performs better than StyelGAN2 here. The contrast between IS and FAD suggests that UNAGAN learns to generate samples whose embeddings have similar distribution as the real data, but its output cannot be easily associated with a genre class by the short-chunk CNN classifier. We also see that StyleGAN has fairly high FAD, showing that its generation hardly resemble the real data distribution.

Out of curiosity, we also train the models on the private, yet larger, *looperman* dataset and redo the evaluation. Table 2 shows that StyleGAN2 achieves even higher IS and much lower NDB here. Furthermore, its FAD is now lower than that of UNAGAN. Together with the result in JS and NDB, we see from this table that StyleGAN2 is more effective in learning to cover the modes in a large dataset. Figure 1 demonstrates the mel-spectrograms of some random drum loops generated by this StyleGAN2 model.

6.2 Subjective Evaluation & Its Result

We run additionally an online listening test to evaluate the models subjectively. Each subject is presented with the a randomly-picked human-made loop from the freesound dataset, and one randomly-generated loop by each of the

[15] https://github.com/ciaua/unagan

	IS ↑	FAD ↓	JS ↓	NDB/K ↓
Looperman	11.9±3.21	0.11	0.01	0.01
Freesound	6.30±1.82	0.72	0.08	0.46
StyleGAN	1.31±1.95	13.78	0.43	0.94
StyleGAN2	**5.24±1.84**	7.91	**0.09**	**0.59**
UNAGAN	3.33±1.65	**4.32**	0.16	0.73

Table 1. Objective evaluation result for the three models trained on the *freesound* dataset. We also display the IS of the two sets of real data. (↓/↑: the lower/higher the better).

	IS ↑	FAD ↓	JS ↓	NDB/K ↓
StyleGAN	1.30±2.00	12.98	0.41	0.87
StyleGAN2	**6.08±2.26**	**2.22**	**0.01**	**0.08**
UNAGAN	3.83±1.72	3.36	0.29	0.89

Table 2. Objective evaluation result for the three models trained on instead the private *looperman* dataset.

three models trained on *freesound*, with the ordering of these four loops randomized. Then, the subject is asked to rate each of these one-bar loops in terms of the following metrics, the first three on a three-point scale, and the last one on a five-point Likert scale:

- **Drumness**: whether the sample contains drum sounds ('no'/'yes but vague'/'yes and clear');
- **Loopness**: whether the sample can be played repeatedly in a seamless manner ('no'/'yes but not so good'/'yes');
- **Audio quality**: whether the sample is free of unpleasant noises or artifacts ('no'/'no but not so bad'/'yes');
- **Preference**: how much you like it (1–5).

To evaluate loopness, we actually repeat each sample four times in the audio recording presented to the subjects. And, since the output of the models go through the MelGAN vocoder to become waveforms, we compute the Mel spectrograms of the human-made loops and render them to audio with the same vocoder for fair comparison.

140 anonymous subjects from Taiwan participated in this test, [16] with in total six unique samples by each model evaluated. Overall, the responses indicated an acceptable level of reliability (Cronbach's $\alpha = 0.709$). We see from Figure 5 that the result of this subjective evaluation is well aligned with that of the objective evaluation, with StyleGAN2 performing the best and StyleGAN the worst, demonstrating the effectiveness of the objective metrics to some extent. Interestingly, we see no statistical difference in the ratings of the StyleGAN2 loops and the (MelGAN-vocoded) freesound loops in Drumness and Preference.

Finally, we correlate the scores of the objective metrics and subjective metrics for the 18 samples evaluated in the listening test (i.e., six samples by each GAN model). We found 0.25–0.37 correlation between IS and the four subjective metrics, and 0.01–0.16 negative correlation between FAD and the subjective metrics. The strongest correlation (0.37) is found between IS and Preference.

[16] The subjects have no ideas about our models beforehand; they neither know that one of the loops they hear is human-made.

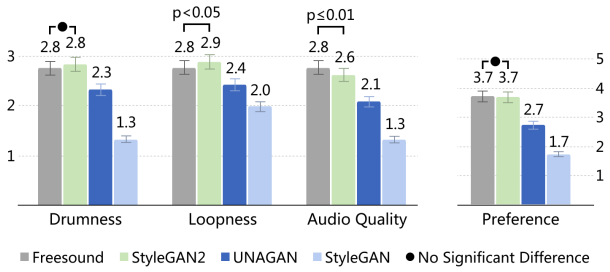


Figure 5. Subjective evaluation result for the three models trained on *freesound*. The performance difference between any pair of models in any metric is statistically significant (p-value< 0.001) under the Wilcoxon signed-rank test, except for the pairs that are explicitly highlighted.

7. CONCLUSION AND FUTURE WORK

In this paper, we have proposed using loop generation as a benchmarking task to provide a standardized evaluation of audio-domain music generation models, taking advantage of the public availability of the large collection of loops in FSLD. Moreover, we developed customized metrics to objectively evaluate the performance of such generative models for the particular case of one-bar drum loops with 120 BPM. As references, we implemented and evaluated three recent model architectures using the dataset, and discovered that StyleGAN2 works quite well. The list of models we have evaluated is short and by no means exhaustive. We wish researchers can find this benchmark useful and consider it as part of their evaluation of new models.

This work can be extended in many other directions. First and foremost, we can extend the benchmark to cover all the loops in FSLD (and looperman). The major complexity here could be the challenge to build a model that fits it all; we may need separate generative models and vocoders for different types of loops.

Second, we are certainly interested in the case of generating loops that have different tempos, rather than a fixed tempo at 120. This will require the generative models to be capable of generating variable-length output, which seems more realistic in musical audio applications.

We can also extend the benchmark to generate four-bar loops (which are not simply repeating a one-bar loop quadruple times), as there are actually a big collection of 6,656 four-bar drum loops in the looperman dataset. We do not evaluate this in this paper, as the public freesound dataset does not contain many such four-bar loops.

We also want to include more objective metrics in the future, such as using the Audio Commons Audio Extractor [34] to evaluate the "loopness" of the generated samples, or using an automatic drum transcription model [51–53] to assess the plausibility of the created percussive patterns.

Besides the benchmarking initiative, we are interested in further improving audio-domain loop generation itself and exploring new use cases, e.g., to have a conditional generation model that gives users some control (in similar veins to [3, 4, 26]), or to aim at generating novel loops by means of a creative adversarial network (CAN) [30].

8. ACKNOWLEDGEMENTS

This research work is supported by the Ministry of Science and Technology (MOST), Taiwan, under grant number 109-2628-E-001-002-MY2.

9. REFERENCES

[1] J. Engel *et al.*, "Neural audio synthesis of musical notes with WaveNet autoencoders," in *Proc. Int. Conf. Machine Learning*, 2017.

[2] J. Engel, K. K. Agrawal, S. Chen, I. Gulrajani, C. Donahue, and A. Roberts, "GANSynth: Adversarial neural audio synthesis," in *Proc. Int. Conf. Learning Representations*, 2019.

[3] J. Nistal, S. Lattner, and G. Richard, "DrumGAN: Synthesis of drum sounds with timbral feature conditioning using generative adversarial networks," in *Proc. Int. Soc. Music Information Retrieval Conf.*, 2020.

[4] A. Ramires, P. Chandna, X. Favory, E. Gómez, and X. Serra, "Neural percussive synthesis parameterised by high-level timbral features," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing*, 2020.

[5] A. van den Oord *et al.*, "WaveNet: A generative model for raw audio," *arXiv preprint arXiv:1609.03499*, 2016.

[6] S. Mehri *et al.*, "SampleRNN: An unconditional end-to-end neural audio generation model," in *Proc. Int. Conf. Learning Representations*, 2017.

[7] C. J. Carr and Z. Zukowski, "Generating albums with SampleRNN to imitate metal, rock, and punk bands," *arXiv preprint:1811.06633*, 2018.

[8] S. Vasquez and M. Lewis, "MelNet: A generative model for audio in the frequency domain," *arXiv preprint arXiv:1906.01083*, 2019.

[9] J.-Y. Liu, Y.-H. Chen, Y.-C. Yeh, and Y.-H. Yang, "Unconditional audio generation with generative adversarial networks and cycle regularization," in *Proc. INTERSPEECH*, 2020.

[10] ——, "Score and lyrics-free singing voice generation," in *Proc. Int. Conf. Computational Creativity*, 2020.

[11] P. Dhariwal, H. Jun, C. Payne, J. W. Kim, A. Radford, and I. Sutskever, "Jukebox: A generative model for music," *arXiv preprint:2005.00341*, 2020.

[12] K. van den Broek, "MP3net: coherent, minute-long music generation from raw audio with a simple convolutional GAN," *arXiv preprint arXiv:2101.04785*, 2021.

[13] T. Karras *et al.*, "Analyzing and improving the image quality of StyleGAN," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2020.

[14] M. Won, A. Ferraro, D. Bogdanov, and X. Serra, "Evaluation of CNN-based automatic music tagging models," in *Proc. Sound and Music Computing Conf.*, 2020.

[15] A. Ramires *et al.*, "The Freesound Loop Dataset and annotation tool," in *Proc. Int. Soc. Music Information Retrieval Conf.*, 2020.

[16] G. Stillar, "Loops as genre resources," *Folia Linguistica*, vol. 39, no. 1-2, pp. 197 – 212, 2005. [Online]. Available: https://www.degruyter.com/view/journals/flin/39/1-2/article-p197.xml

[17] J. Ching, A. Ramires, and Y.-H. Yang, "Instrument role classification: Auto-tagging for loop based music," in *Proc. Joint Conference on AI Music Creativity*, 2020.

[18] I. J. Goodfellow *et al.*, "Generative adversarial nets," in *Proc. Advances in Neural Information Processing Systems*, 2014, pp. 2672–2680.

[19] T. Karras, S. Laine, and T. Aila, "A style-based generator architecture for generative adversarial networks," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2019, pp. 4396–4405.

[20] B.-Y. Chen, J. Smith, and Y.-H. Yang, "Neural loop combiner: Neural network models for assessing the compatibility of loops," in *Proc. Int. Soc. Music Information Retrieval Conf.*, 2020.

[21] R. Vogl and P. Knees, "An intelligent drum machine for Electronic Dance Music production and performance," in *Proc. Int. Conf. New Interfaces for Musical Expression*, 2017.

[22] T. Salimans *et al.*, "Improved techniques for training GANs," in *Proc. Conf. Neural Information Processing Systems*, 2016, pp. 2226–2234.

[23] J. Nistal, C. Aouameur, S. Lattner, and G. Richard, "VQCPC-GAN: Variable-length adversarial audio synthesis using vector-quantized contrastive predictive coding," in *Proc. IEEE Workshop on Applications of Signal Processing to Audio and Acoustics*, 2021.

[24] C. Aouameur, P. Esling, and G. Hadjeres, "Neural drum machine: An interactive system for real-time synthesis of drum sounds," *arXiv preprint arXiv:1907.02637*, 2019.

[25] J. Drysdale, M. Tomczak, and J. Hockman, "Adversarial synthesis of drum sounds," in *Proc. Int. Conf. Digital Audio Effects*, 2020.

[26] P. Chandna, A. Ramires, X. Serra, and E. Gómez, "LoopNet: Musical loop synthesis conditioned on intuitive musical parameters," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing*, 2021.

[27] J. Gillick, A. Roberts, J. Engel, D. Eck, and D. Bamman, "Learning to groove with inverse sequence transformations," in *Proc. Int. Conf. Machine Learning*, 2019.

[28] V. Thio, H.-M. Liu, Y.-C. Yeh, and Y.-H. Yang, "A minimal template for interactive web-based demonstrations of musical machine learning," in *Proc. Workshop on Intelligent Music Interfaces for Listening and Creation*, 2019.

[29] G. Alain, M. Chevalier-Boisvert, F. Osterrath, and R. Piche-Taillefer, "DeepDrummer: Generating drum loops using deep learning and a human in the loop," *arXiv preprint arXiv:2008.04391*, 2020.

[30] N. Tokui, "Can GAN originate new electronic dance music genres? – Generating novel rhythm patterns using GAN with genre ambiguity loss," *arXiv preprin: 2011.13062*, 2020.

[31] H.-W. Dong, W.-Y. Hsiao, L.-C. Yang, and Y.-H. Yang, "MuseGAN: Symbolic-domain music generation and accompaniment with multi-track sequential generative adversarial networks," in *Proc. AAAI Conf. Artificial Intelligence*, 2018.

[32] I. Simon, A. Roberts, C. Raffel, J. Engel, C. Hawthorne, and D. Eck, "Learning a latent space of multitrack measures," *arXiv preprint arXiv:1806.00195*, 2018.

[33] Y. Ren, J. He, X. Tan, T. Qin, Z. Zhao, and T.-Y. Liu, "PopMAG: Pop music accompaniment generation," in *Proc. ACM Multimedia Conf.*, 2020.

[34] F. Font, T. Brookes, G. Fazekas, M. Guerber, A. La Burthe, D. Plans, M. Plumbley, W. Wang, and X. Serra, "Audio Commons: Bringing Creative Commons audio content to the creative industries," in *Proc. AES Int. Conf. Audio for Games,*, 2016.

[35] S. Lattner and M. Grachten, "High-level control of drum track generation using learned patterns of rhythmic interaction," in *Proc. IEEE Work. Applications of Signal Processing to Audio and Acoustics*, 2019.

[36] S. Barratt and R. Sharma, "A note on the inception score," in *Proc. ICML Works. Theoretical Foundations and Applications of Deep Generative Models*, 2018.

[37] S. Böck, F. Korzeniowski, J. Schlüter, F. Krebs, and G. Widmer, "Madmom: A new Python audio and music signal processing library," in *Proc. ACM Multimedia Conf.*, 2016.

[38] S. Böck, F. Krebs, and G. Widmer, "Joint beat and downbeat tracking with recurrent neural networks," in *Proc. Int. Soc. Music Information Retrieval Conf.*, 2016.

[39] F. Gouyon, A. Klapuri, S. Dixon, M. Alonso, G. Tzanetakis, C. Uhle, and P. Cano, "An experimental comparison of audio tempo induction algorithms," *IEEE Trans. Audio, Speech, and Language Processing*, vol. 14, no. 5, pp. 1832–1844, 2006.

[40] K. Kilgour, M. Zuluaga, D. Roblek, and M. Sharifi, "Fréchet Audio Distance: A metric for evaluating music enhancement algorithms," *arXiv preprint arXiv: 1812.08466*, 2019.

[41] E. Richardson and Y. Weiss, "On GANs and GMMs," in *Proc. Conf. Neural Information Processing Systems*, 2018.

[42] T. Karras, T. Aila, S. Laine, and J. Lehtinen, "Progressive growing of GANs for improved quality, stability, and variation," in *Proc. Int. Conf. Learning Representations*, 2018.

[43] K. Palkama, L. Juvela, and A. Ilin, "Conditional spoken digit generation with StyleGAN," in *Proc. INTERSPEECH*, 2020.

[44] R. Prenger, R. Valle, and B. Catanzaro, "WaveGlow: A flow-based generative network for speech synthesis," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing*, 2019.

[45] Z. Kong, W. Ping, J. Huang, K. Zhao, and B. Catanzaro, "DiffWave: A versatile diffusion model for audio synthesis," in *Proc. Int. Conf. Learning Representations*, 2021.

[46] K. Kumar, R. Kumar, T. de Boissiere, L. Gestin, W. Z. Teoh, J. Sotelo, A. de Brebisson, Y. Bengio, and A. Courville, "MelGAN: Generative adversarial networks for conditional waveform synthesis," *arXiv preprint arXiv:1910.06711*, 2019.

[47] L. Mescheder, A. Geiger, and S. Nowozin, "Which training methods for gans do actually converge?" *arXiv preprint arXiv:1801.04406*, 2018.

[48] C. Donahue, J. McAuley, and M. Puckette, "Adversarial audio synthesis," in *Proc. Int. Conf. Learning Representations*, 2019.

[49] D. Berthelot, T. Schumm, and L. Metz, "BEGAN: Boundary equilibrium generative adversarial networks," *arXiv preprint:1703.10717*, 2017.

[50] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, "Unpaired image-to-image translation using cycle-consistent adversarial networks," in *Proceedings of the IEEE international conference on computer vision*, 2017.

[51] C.-W. Wu, C. Dittmar, C. Southall, R. Vogl, G. Widmer, J. Hockman, M. Müller, and A. Lerch, "A review of automatic drum transcription," *IEEE/ACM Trans. Audio, Speech, and Language Processing*, vol. 26, no. 9, pp. 1457–1483, 2018.

[52] K. Choi and K. Cho, "Deep unsupervised drum transcription," in *Proc. Int. Soc. Music Information Retrieval Conf.*, 2020.

[53] L. Callender, C. Hawthorne, and J. H. Engel, "Improving perceptual quality of drum transcription with the expanded groove MIDI dataset," *arXiv preprint:2004.00188*, 2020.